

Table des matières

Labo 07 — CI/CD et empaquetage du portfolio	1
Point de départ	2
Étape 1 — sqlfluff : le SQL aux conventions	3
Étape 2 — La cible ci et son schéma jetable	4
Étape 3 — Les secrets GitHub	5
Étape 4 — Le workflow GitHub Actions	6
Étape 5 — Voir la CI échouer, puis passer	7
Étape 6 — Fraîcheur et sévérité.....	8
Étape 7 — Le README du projet	10
Étape 8 — Publier le dépôt	11
Récapitulatif	12
Dépannage	12
Annexe — les fichiers de l'étape, d'un tenant.....	13

Labo 07 — CI/CD et empaquetage du portfolio

Module 7/8 — Formation Analytics Engineering

Support d'avant la refonte. La formation a été refondue le 2026-09-20, de Snowflake vers DuckDB et dbt-duckdb, sur un univers Boréal redessiné. Ce cours porte encore l'ancienne pile et l'ancienne numérotation : il sert de matière au futur Cours 08, qui n'est pas encore écrit. Les cours 01 à 06 du dossier, eux, sont à jour. Voir README.md.

Ce que vous allez faire. Entourer le pipeline d'un filet automatique, puis rendre le dépôt présentable :

- installer sqlfluff et mettre le SQL du projet aux conventions ;
- créer une cible ci et un schéma de CI jetable dans Snowflake ;
- écrire un workflow GitHub Actions qui lance le lint puis dbt build à chaque *pull request* ;
- ranger les identifiants dans les secrets GitHub, sans rien mettre dans le dépôt ;

- ajouter un contrôle de fraîcheur des sources et graduer la sévérité d'un test ;
- remplacer le README par défaut, exporter les captures, publier le dépôt ;
- fabriquer une *pull request* qui échoue, la corriger, la voir passer.

À la fin, Boréal est une pièce de portfolio : un dépôt public, avec une CI verte, un rapport et un README qui se lit en deux minutes.

Prérequis : Labos 00 à 06 faits, `dbt build --target prod` au vert, rapport Power BI enregistré dans `Projet_Boreal\rapport\`. Un compte GitHub. Environnement `conda` formations actif.

La liste de contrôle du cours :

- ☐ `sqlfluff` installé, `.sqlfluff` configuré, `sqlfluff lint models/` propre
- ☐ Schéma de CI et cible `ci` en place ; `ci/profiles.yml` versionné, sans aucune valeur
- ☐ Secrets GitHub renseignés (compte, utilisateur, mot de passe, rôle, entrepôt)
- ☐ `.github/workflows/ci.yml` écrit ; une *pull request* le déclenche
- ☐ Une *pull request* volontairement fautive : CI rouge, puis verte après correction
- ☐ `dbt source freshness` configuré sur les 5 sources
- ☐ Un test passé en `warn` avec un seuil justifié
- ☐ README du projet réécrit, captures en place, PDF du rapport présent
- ☐ Dépôt publié sur GitHub, sans secret dans l'historique

Point de départ

Le pipeline est complet et il tourne. Ce qui manque n'est plus du pipeline : c'est ce qui empêche un futur changement de le casser en silence, et ce qui permet à quelqu'un d'autre de comprendre ce que vous avez fait.

Tout ce labo se passe dans `Projet_Boreal`, le dépôt Git séparé, celui qui deviendra public. Les chemins sont relatifs à cette racine, sauf mention contraire ; les fichiers `dbt` (`models/`, `dbt_project.yml`) sont, comme toujours, sous `Projet_Boreal\boreal\`.

Piège — deux dépôts, encore. La formation a son dépôt, votre projet a le sien. Tout ce qui suit (le workflow, le README, les secrets) appartient au vôtre. Lancez `git` depuis `Projet_Boreal`, jamais depuis un dossier de cours.

Étape 1 — sqlfluff : le SQL aux conventions

Théorie : Theorie_07.md, section 6.

1. Installez l'outil dans l'environnement formations :

```
pip install sqlfluff sqlfluff-templater-dbt
```

2. À la racine du projet dbt (Projet_Boreal\boreal\), créez .sqlfluff :

```
[sqlfluff]
dialect = snowflake
templater = dbt
max_line_length = 120
exclude_rules = ST06, RF04
```

```
[sqlfluff:indentation]
tab_space_size = 4
```

```
[sqlfluff:rules:capitalisation.keywords]
capitalisation_policy = lower
```

```
[sqlfluff:rules:capitalisation.identifiers]
extended_capitalisation_policy = lower
```

Le `templater = dbt` est la ligne qui compte : sans elle, `sqlfluff` bute sur les `{{ ref(...) }}` qu'il ne sait pas lire. Les deux règles exclues concernent l'ordre des colonnes et les mots réservés, elles génèrent surtout du bruit sur un projet dbt.

3. Lancez le lint, puis la correction automatique :

```
sqlfluff lint models/
sqlfluff fix models/
```

`fix` corrige seul l'indentation, la casse et les espaces. Relisez son diff avant de committer : il ne se trompe presque jamais, mais « presque » suffit à justifier un coup d'œil.

4. Relancez `sqlfluff lint models/` jusqu'à ce qu'il ne signale plus rien, puis vérifiez que le projet construit toujours :

```
dbt build
```

Pourquoi le style se règle par un outil. Une convention écrite dans un document est négociée à chaque revue ; une convention écrite dans un fichier de configuration ne l'est plus. Ça libère la revue humaine pour ce qu'elle fait le mieux, le grain, la modélisation, la réutilisabilité (Théorie 07, section 1).

Étape 2 — La cible `ci` et son schéma jetable

Théorie : *Theorie_07.md*, section 4.

La CI doit pouvoir écrire dans Snowflake sans jamais toucher la production.

1. Dans Snowsight, donnez au rôle du projet le droit de créer ses schémas de CI, il l'a déjà, depuis le Labo 01 (`CREATE SCHEMA ON DATABASE BOREAL`). Vérifiez simplement :

```
USE ROLE BOREAL_ROLE; USE WAREHOUSE BOREAL_WH;
SHOW GRANTS TO ROLE BOREAL_ROLE;
```

Aucun schéma n'est à créer à la main : dbt créera `CI_STAGING`, `CI_MARTS` et `CI_PRESENTATION` tout seul, grâce à la macro du Cours 03 et à la cible qu'on ajoute ci-dessous.

2. Créez `ci/profiles.yml` — à la racine de `Projet_Boreal`, pas dans `boreal\` :

```
boreal:
  target: ci
  outputs:
    ci:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      role: "{{ env_var('SNOWFLAKE_ROLE') }}"
      warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
      database: BOREAL
      schema: CI
      threads: 4
```

Ce fichier se versionne, contrairement à celui de votre poste. Relisez-le : il ne contient aucune valeur, seulement des noms de variables. C'est la distinction de la Théorie 07, section 5, un fichier de références, pas un fichier de secrets.

3. Le `schema: CI` est ce qui produira `CI_STAGING`, `CI_MARTS`, `CI_PRESENTATION` : la macro `generate_schema_name`, rendue consciente de l'environnement au Cours 05, préfixe par le schéma de la cible pour tout ce qui n'est pas prod. Aucun modèle ne change.
4. Essayez la cible depuis votre poste, en pointant dbt sur ce profil :

```
cd boreal
dbt build --profiles-dir ../ci --target ci
```

Les variables SNOWFLAKE_ACCOUNT, SNOWFLAKE_USER, SNOWFLAKE_ROLE et SNOWFLAKE_WAREHOUSE doivent être définies dans votre terminal, vous n’aviez posé que SNOWFLAKE_PASSWORD au Cours

- 5. Ajoutez-les de la même façon (set pour la session, setx pour de bon).
- 6. Vérifiez dans Snowsight que les schémas de CI existent, puis faites le ménage :

```
SHOW SCHEMAS IN DATABASE BOREAL;          -- CI_STAGING, CI_MARTS, CI_PRESENTATION

DROP SCHEMA IF EXISTS BOREAL.CI_STAGING;
DROP SCHEMA IF EXISTS BOREAL.CI_MARTS;
DROP SCHEMA IF EXISTS BOREAL.CI_PRESENTATION;
```

Ces schémas sont jetables par construction : la CI les recrée à chaque exécution.

Étape 3 — Les secrets GitHub

Théorie : Theorie_07.md, section 5.

- 1. Créez le dépôt distant, s’il n’existe pas encore : sur GitHub, Nouveau dépôt, nommé projet-boreal (ou le nom de votre choix), public, sans README ni .gitignore, vous avez déjà les vôtres.
- 2. Dans le dépôt : Settings → Secrets and variables → Actions → New repository secret. Créez les cinq secrets :

Nom	Valeur
SNOWFLAKE_ACCOUNT	votre identifiant de compte, format orgname-account_name
SNOWFLAKE_USER	votre utilisateur Snowflake
SNOWFLAKE_PASSWORD	votre mot de passe
SNOWFLAKE_ROLE	BOREAL_ROLE
SNOWFLAKE_WAREHOUSE	BOREAL_WH

GitHub chiffre ces valeurs et les masque dans les journaux d’exécution. Personne, y compris vous, ne peut les relire ensuite, seulement les remplacer.

Piège — le dépôt public et l’historique. Avant le premier git push, relisez ce que vous allez envoyer : git log -p | grep -i "password\|account" sur les fichiers de configuration. Un secret poussé puis retiré reste dans l’historique, et sur un dépôt public il faut le considérer comme brûlé (Théorie 07, section 5).

Étape 4 — Le workflow GitHub Actions

Théorie : Theorie_07.md, section 3.

Créez `.github/workflows/ci.yml`, à la racine de `Projet_Boreal` :

```
name: CI dbt

on:
  pull_request:
    branches: [ master ]

jobs:
  qualite:
    runs-on: ubuntu-latest

    env:
      SNOWFLAKE_ACCOUNT:  ${ secrets.SNOWFLAKE_ACCOUNT }
      SNOWFLAKE_USER:     ${ secrets.SNOWFLAKE_USER }
      SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }
      SNOWFLAKE_ROLE:     ${ secrets.SNOWFLAKE_ROLE }
      SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }

    steps:
      - name: Récupérer le dépôt
        uses: actions/checkout@v4

      - name: Installer Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Installer dbt et sqlfluff
        run: pip install dbt-snowflake sqlfluff sqlfluff-templater-dbt

      - name: Installer les paquets dbt
        working-directory: boreal
        run: dbt deps --profiles-dir ../ci

      - name: Lint du SQL
        working-directory: boreal
        run: sqlfluff lint models/ --profiles-dir ../ci
```

```

- name: Construire et tester sur le schéma de CI
  working-directory: boreal
  run: dbt build --profiles-dir ../ci --target ci

- name: Nettoyer les schémas de CI
  if: always()
  working-directory: boreal
  run: dbt run-operation drop_ci_schemas --profiles-dir ../ci --target ci

```

Quelques points à comprendre plutôt qu'à copier.

Le déclencheur est `pull_request` : le workflow ne tourne pas à chaque commit, mais à chaque proposition d'intégration. Remplacez `master` par `main` si c'est le nom de votre branche principale.

Le bloc `env` : traduit les secrets GitHub en variables d'environnement, celles-là mêmes que `ci/profiles.yml` va lire par `env_var(...)`. C'est le point de jonction entre les deux moitiés du dispositif.

Les trois étapes d'installation existent parce que le *runner* est neuf : il n'a ni Python configuré, ni dbt, ni vos paquets (Théorie 07, section 3).

Le `if: always()` de la dernière étape fait tourner le nettoyage même quand la construction a échoué, sinon un échec laisserait des schémas derrière lui à chaque fois.

Cette dernière étape a besoin d'une macro. Créez `boreal/macros/drop_ci_schemas.sql` :

```

{% macro drop_ci_schemas() %}
  {% if target.name == 'ci' %}
    {% for suffixe in ['STAGING', 'MARTS', 'PRESENTATION'] %}
      {% set requete %}
        drop schema if exists {{ target.database }}.CI_{{ suffixe }} cascade
      {% endset %}
      {% do run_query(requete) %}
      {{ log("Schéma CI_" ~ suffixe ~ " supprimé", info=True) }}
    {% endfor %}
  {% endif %}
{% endmacro %}

```

Le garde-fou `if target.name == 'ci'` est ce qui empêche cette macro de faire des dégâts si quelqu'un la lance par mégarde sur dev ou prod. Une macro qui supprime des schémas doit toujours porter ce genre de verrou.

Étape 5 — Voir la CI échouer, puis passer

Théorie : Theorie_07.md, section 2.

Une CI qu'on n'a jamais vue rouge est une CI dont on ne sait rien.

1. Créez une branche et cassez volontairement quelque chose :

```
git checkout -b essai-ci
```

2. Dans `boreal/models/marts/dim_produit.sql`, retirez la ligne qui génère la clé de substitution, le test unique sur `produit_key` n'aura alors plus de colonne à contrôler, et la construction échouera.
3. Poussez et ouvrez la *pull request* :

```
git add boreal/models/marts/dim_produit.sql
git commit -m "Essai : casser dim_produit pour voir la CI rouge"
git push -u origin essai-ci
```

Sur GitHub, le bouton « Compare & pull request » apparaît. Ouvrez-la.

4. Regardez l'onglet Checks : le workflow démarre, installe, puis échoue. Ouvrez le journal de l'étape fautive et lisez le message, c'est exactement ce qu'aurait vu un collègue.
5. Corrigez, committez, poussez sur la même branche : le workflow se relance tout seul et passe au vert. Fusionnez la *pull request*, puis supprimez la branche.

Ce que vous venez de démontrer. Un changement qui casse le modèle n'atteint jamais la branche principale. C'est tout l'objet du module, et c'est aussi ce qu'on montre en entrevue : une capture de la CI rouge puis verte vaut mieux qu'un paragraphe sur la rigueur.

Étape 6 — Fraîcheur et sévérité

Théorie : Theorie_07.md, section 6.

Deux réglages qui élargissent la qualité au-delà des tests de cohérence.

1. La fraîcheur des sources. Ouvrez `boreal/models/staging/_sources.yml` et ajoutez un bloc `freshness` au niveau de la source, plus la colonne qui porte l'horodatage de chargement :

```
sources:
- name: raw
  database: BOREAL
  schema: RAW
  description: "Extraits SAP bruts de Boréal, chargés tels quels (Cours 01)."
```

```
loaded_at_field: _CHARGE_LE
freshness:
  warn_after: { count: 36, period: hour }
  error_after: { count: 72, period: hour }
tables:
  - name: t005s
    freshness: null      # table de référence : elle ne change presque jamais
  - name: kna1
    freshness: null
  - name: mara
    freshness: null
  - name: vbak
  - name: vbap
```

Les tables de référence sont exclues par freshness: null : leur immobilité est normale, les surveiller ne produirait que du bruit. Seules les deux tables de commandes doivent bouger.

2. La colonne _CHARGE_LE n'existe pas encore dans vos tables RAW, le Labo 01 chargeait sans horodatage. Ajoutez-la, avec une valeur par défaut, puis remplissez-la pour l'existant :

```
USE ROLE BOREAL_ROLE; USE WAREHOUSE BOREAL_WH;
```

```
ALTER TABLE BOREAL.RAW.VBAK ADD COLUMN _CHARGE_LE TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP();
ALTER TABLE BOREAL.RAW.VBAP ADD COLUMN _CHARGE_LE TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP();
```

```
UPDATE BOREAL.RAW.VBAK SET _CHARGE_LE = CURRENT_TIMESTAMP() WHERE _CHARGE_LE IS NULL;
UPDATE BOREAL.RAW.VBAP SET _CHARGE_LE = CURRENT_TIMESTAMP() WHERE _CHARGE_LE IS NULL;
```

Le préfixe _ marque une colonne technique, ajoutée par le chargement et non par la source. Les modèles de staging ne la sélectionnent pas : elle n'a rien à faire en aval.

3. Lancez le contrôle :

```
dbt source freshness
```

Il doit passer au vert : vous venez de recharger l'horodatage. Attendez 36 heures sans rien charger et il passerait en avertissement : c'est le seul test qui attrape une table figée pendant que tous les autres restent verts.

4. La sévérité graduée. Dans boreal/models/marts/_marts.yml, passez un test tolérant en avertissement avec un seuil, par exemple l'année de modèle, absente pour les pièces :

```
- name: annee_modele
  description: "Année du modèle. Nulle pour les pièces et accessoires – attendu."
```

```
tests:
  - not_null:
    config:
      severity: warn
      warn_if: ">50"
      error_if: ">200"
```

La lecture : en dessous de 50 lignes nulles, silence ; au-delà, un avertissement ; au-delà de 200, un échec. On documente le bruit connu au lieu de supprimer le test, c'est ce qui empêche une équipe de prendre l'habitude d'ignorer le rouge (Théorie 07, section 6).

5. Relancez `dbt build` pour vérifier que tout tient, puis committez.

Étape 7 — Le README du projet

Théorie : Theorie_07.md, section 8.

C'est le fichier que le plus de gens liront, et celui que `dbt init` a laissé en anglais et générique. Remplacez `boreal/README.md`, ou mieux, écrivez le vrai README à la racine de `Projet_Boreal`, qui est ce que GitHub affiche.

La trame qui fonctionne, dans cet ordre :

Section	Ce qu'on y met
Titre et une phrase	ce que fait le projet, sans jargon
Le problème	d'où part la donnée : extraits ERP, noms techniques, dates en texte
Le résultat	une capture d'une page du rapport Power BI
L'architecture	RAW → STAGING → INTERMEDIATE → MARTS → PRESENTATION → BI, en une ligne, puis un tableau des couches
La pile	Snowflake, dbt, Power BI, GitHub Actions
Le lignage	la capture du graphe de <code>dbt docs</code>
Rejouer le projet	les commandes, dans l'ordre, en partant de zéro
Ce que le projet démontre	grain testé, lignage, CI, séparation dev/prod
Limites assumées	données générées, déploiement manuel

Produisez les deux captures :

```
cd boreal
dbt docs generate
dbt docs serve
```

Le site s'ouvre dans le navigateur ; cliquez sur l'icône de lignage en bas à droite et capturez le graphe complet. Rangez l'image dans `Projet_Boreal\images\`, avec la capture du rapport Power BI. Le PDF du rapport, exporté au Cours 06, va dans `rapport\`.

Piège — les captures et vos identifiants. Une capture de `dbt docs` ou de Snowsight montre souvent l'identifiant de compte, votre nom d'utilisateur ou un chemin local. Masquez-les avant de pousser : le dépôt est public, et une image ne se retire pas de l'historique plus facilement qu'un mot de passe.

Sur la reproductibilité, tranchez et écrivez votre choix dans le README. Le brut n'est pas versionné ; sans le générateur, personne ne peut rejouer le projet. Copier `Cours/generer_sources.py` à la racine de `Projet_Boreal` rend la pièce autonome. Si vous le faites, dites dans le README que l'original vit avec la formation et que cette copie est un artefact d'empaquetage, c'est ce qui évite qu'on se demande un jour lequel fait foi.

Étape 8 — Publier le dépôt

Théorie : Theorie_07.md, sections 1 et 8.

1. Dernière vérification avant le premier envoi :

```
git status
git log --oneline
```

Personne ne doit voir `sources_sap/`, `.env`, ni un `profiles.yml` de poste dans la liste des fichiers suivis. Le `ci/profiles.yml` s'y trouve, lui, et c'est voulu.

2. Reliez et poussez :

```
git remote add origin https://github.com/<votre_compte>/projet-boreal.git
git branch -M master
git push -u origin master
```

3. Sur GitHub, vérifiez que le README s'affiche correctement, que les images se chargent, et que l'onglet Actions montre l'exécution verte de votre *pull request*.
4. Dans les paramètres du dépôt, ajoutez une description et des sujets (`dbt`, `snowflake`, `analytics-engineering`, `power-bi`) : c'est ce par quoi le dépôt se trouve.

Récapitulatif

- ☐ sqlfluff configuré, `models/` propre.
- ☐ Cible `ci` et macro de nettoyage en place ; `ci/profiles.yml` sans aucune valeur.
- ☐ Cinq secrets GitHub renseignés.
- ☐ Workflow déclenché à chaque *pull request* : lint, dbt build, nettoyage.
- ☐ Une *pull request* vue rouge puis verte.
- ☐ dbt source freshness configuré ; un test gradué en warn avec seuils.
- ☐ README réécrit, deux captures, PDF du rapport.
- ☐ Dépôt public, historique sans secret.

La formation est terminée. Vous avez monté une plateforme analytique complète (de l'extrait brut au rapport), testée, documentée, versionnée et protégée par une intégration continue. Ce qui reste à faire n'est plus technique : c'est de savoir la raconter en cinq minutes, et la Théorie 07, question à développer B, en donne le plan.

Dépannage

Symptôme	Cause probable	Correctif
sqlfluff échoue sur <code>{{ ref(...) }}</code>	<code>templater = dbt</code> absent du <code>.sqlfluff</code> , ou <code>sqlfluff-templater-dbt</code> non installé	corriger la config, réinstaller le paquet
sqlfluff ne trouve pas le profil	il compile la Jinja, donc il lui faut une connexion	ajouter <code>--profiles-dir ../ci</code> comme dans le workflow
Le workflow ne se déclenche pas	branche du on : <code>pull_request</code> $\neq$ nom réel de la branche principale	remplacer <code>master</code> par <code>main</code> , ou l'inverse
Could not find profile named 'boreal' en CI	mauvais <code>--profiles-dir</code> , ou <code>working-directory</code> oublié	le chemin est relatif au dossier de travail de l'étape
Erreur d'authentification en CI, jamais en local	un secret manquant ou mal nommé	les noms des secrets doivent correspondre exactement au bloc env :

Symptôme	Cause probable	Correctif
La CI écrit dans DEV_MARTS	--target ci oublié dans la commande	ajouter la cible ; vérifier schema: CI dans ci/profiles.yml
Des schémas CI_* s'accumulent	l'étape de nettoyage n'a pas tourné	vérifier le if: always() et le garde-fou de la macro
dbt source freshness échoue : colonne introuvable	loaded_at_field absent des tables RAW	ajouter la colonne _CHARGE_LE (étape 6)
Les images du README ne s'affichent pas sur GitHub	chemin absolu, ou dossier non committé	chemin relatif (images/...), et vérifier que le dossier est suivi

Annexe — les fichiers de l'étape, d'un tenant

boreal/.sqlfluff:

```
[sqlfluff]
dialect = snowflake
templater = dbt
max_line_length = 120
exclude_rules = ST06, RF04
```

```
[sqlfluff:indentation]
tab_space_size = 4
```

```
[sqlfluff:rules:capitalisation.keywords]
capitalisation_policy = lower
```

```
[sqlfluff:rules:capitalisation.identifiers]
extended_capitalisation_policy = lower
```

ci/profiles.yml:

```
boreal:
  target: ci
  outputs:
    ci:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('SNOWFLAKE_USER') }}"
```

```
password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
role: "{{ env_var('SNOWFLAKE_ROLE') }}"
warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
database: BOREAL
schema: CI
threads: 4
```

boreal/macros/drop_ci_schemas.sql:

```
{% macro drop_ci_schemas() %}
  {% if target.name == 'ci' %}
    {% for suffixe in ['STAGING', 'MARTS', 'PRESENTATION'] %}
      {% set requete %}
        drop schema if exists {{ target.database }}.CI_{{ suffixe }} cascade
      {% endset %}
      {% do run_query(requete) %}
      {{ log("Schéma CI_" ~ suffixe ~ " supprimé", info=True) }}
    {% endfor %}
  {% endif %}
{% endmacro %}
```

Les commandes, dans l'ordre :

```
pip install sqlfluff sqlfluff-templater-dbt
sqlfluff fix models/
sqlfluff lint models/
dbt build --profiles-dir ../ci --target ci
dbt source freshness
dbt docs generate
git push -u origin master
```